



线程的暂停

北京理工大学计算机学院
金旭亮

暂停线程的执行



可以使用**Thread.sleep()**方法让线程休眠指定的时间，从而“拖慢”线程的执行过程。

```
public void run() {  
    for(int i = 0; i<10; i++){  
        try {  
            Thread.sleep(500);  
        }  
        catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        //.....  
    }  
}
```

sleep方法的作用相当于执行该方法的线程对线程调度器说：“我想休息一会，过段时间你再叫醒我”。

示例：ThreadSleep.java

另一种暂停线程的方法



使用TimeUnit枚举类型的sleep方法，也能实现暂停，用它的好处是能方便指定使用特定的时间单位而无需人工换算：纳秒、微秒、毫秒、秒、分钟、小时、天.....

```
try {  
    // 休眠1秒钟  
    TimeUnit.SECONDS.sleep(1);  
} catch (InterruptedException e) {  
    System.out.printf("The thread has been interrupted");  
}
```



当线程休眠时，会放弃已获得的CPU，进入TIMED_WAITING状态，休眠结束之后，重新参与线程调度。

用休眠模拟多个并发工作任务

```
RunMutliThread.java x
1 public class RunMutliThread {
2     public static void main(String[] args) {
3         //启动5个线程
4         for (int i = 0; i < 5; i++)
5             createDownloadTask();
6     }
7     //启动一个线程，模拟从网络上下载
8     1 usage
9     private static void createDownloadTask() {
10         Thread downloadThread = new Thread(() -> {
11             var threadName :String = Thread.currentThread().getName();
12             System.out.println(threadName + "正在下载...");
13             try {
14                 Thread.sleep( millis: 5000); //暂停5秒
15             } catch (InterruptedException e) {
16                 e.printStackTrace();
17             }
18             System.out.println(threadName + "下载结束");
19         });
20         downloadThread.start();
21     }
22 }
```

```
Run: RunMutliThread x
"C:\Program Files\Java\jdk-17.0.3.
Thread-4:正在下载...
Thread-0:正在下载...
Thread-1:正在下载...
Thread-2:正在下载...
Thread-3:正在下载...
Thread-0:下载结束
Thread-1:下载结束
Thread-2:下载结束
Thread-3:下载结束
Thread-4:下载结束

Process finished with exit code 0
```

由于5个线程并发运行，所以，并不需要等待 $5 \times 5 = 25$ 秒这个程序才能结束。

实现倒计时计数器



```
Run: CountDownTimer x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
剩余10秒
剩余9秒
剩余8秒
剩余7秒
剩余6秒
剩余5秒
剩余4秒
剩余3秒
剩余2秒
剩余1秒
时间到!

Process finished with exit code 0
```

请使用让线程休眠的方法，
写一个倒计时计数器。

如写不出来，参考：
`CountDownTimer.java`

yield()方法



在Java并发API中，还有一个方法可以使线程释放CPU资源，这就是**yield()**方法。该方法告知JVM当前线程可以为其他任务放弃CPU资源，但JVM并不保证一定会响应该请求。



此方法不会改变线程所处的运行状态。通常只在调试中使用该方法，真正的开发中很少用。

yield方法的作用，相当于执行该方法的线程对线程调度器说：“我现在不急，如果其他人需要CPU的话，先给他用。当然，如果没有其他人要用，我也不介意继续占用。”

sleep vs. yield



- ✓ sleep会导致当前线程暂停指定的时间，没有CPU时间片的消耗。
- ✓ yield只是对CPU调度器的一个提示，如果CPU调度器没有忽略这个提示，它会导致线程上下文的切换。
- ✓ sleep会使线程进入阻塞状态并释放CPU资源。
- ✓ yield会使RUNNING状态的线程进入RUNNABLE状态（如果CPU调度器没有忽略这个提示的话）。
- ✓ 一个线程调用sleep休眠后，另一个线程调用interrupt方法时，它会捕获到中断信号，而yield则不会。

线程的“暂停”与“继续”

```
public class ParkAndUnpark {  
    public static void main(String[] args) throws InterruptedException {  
        //线程一，活干到一半，停下来等通知  
        Thread thread1 = new Thread(() -> {  
            var threadName :String = Thread.currentThread().getName();  
            for (int i = 0; i < 8; i++) {  
                System.out.println(threadName + ",i=" + i);  
                if (i == 3) {  
                    System.out.println(threadName + "在此暂停，等待通知.....");  
                    LockSupport.park();  
                }  
            }  
        });  
        //线程二，向线程一发送通知：“你可以继续干活了.....”  
        Thread thread2 = new Thread(() -> {  
            var threadName :String = Thread.currentThread().getName();  
            System.out.println(threadName + "正在运行");  
            System.out.println(threadName + "发送解锁通知.....");  
            LockSupport.unpark(thread1);  
        });  
    }  
}
```

```
//启动线程一  
thread1.start();  
//主线程休眠2秒  
Thread.sleep( millis: 2000);  
//启动线程二  
thread2.start();  
//等待两个线程的结束  
thread1.join();  
thread2.join();  
}
```


运行截图



The screenshot shows the 'Run' console of an IDE. The title bar indicates the program is 'ParkAndUnpark'. The console output is as follows:

```
Run: ParkAndUnpark x
"C:\Program Files\Java\jdk-17\bin\java.exe"
Thread-0,i=0
Thread-0,i=1
Thread-0,i=2
Thread-0,i=3
Thread-0在此暂停, 等待通知.....
Thread-1正在运行
Thread-1发送解锁通知.....
Thread-0,i=4
Thread-0,i=5
Thread-0,i=6
Thread-0,i=7
Process finished with exit code 0
```

An orange callout box highlights the period between `Thread-0,i=3` and `Thread-0,i=4`, containing the text: 收到通知后, Thread-0 继续运行。

定时任务



使用Timer类的`schedule`方法可以定时执行一个任务。此任务是TimerTask类型的实例（注意TimerTask派生自Runnable），会在另外一个线程中运行这个定时任务。

如果使用RxJava，有一个Observable.interval()方法能更方便地实现定时调用，并且能很方便地切换线程，推荐使用。

```
public class UseTimer {  
  
    public static void main(String[] args) {  
        //将要被定时执行的任务，需要派生自TimerTask抽象类，并实现其抽象方法run()  
        TimerTask task = new TimerTask() {  
            2 usages  
            private int counter = 0;  
  
            public void run() {  
                counter++;  
                System.out.println(Thread.currentThread().getName()  
                    + ":" + counter);  
            }  
        };  
        Timer timer = new Timer();  
        //过2秒钟后首次运行，以后每隔3秒运行一次  
        timer.schedule(task, delay: 2000, period: 3000);  
    }  
}
```

示例：UseTimer.java